

MCTS — Desarrollo de Software Seguro

01. Inicial

Guía de Arranque desde Cero — IOT-Server

IOT-Server Platform

Resumen

Este documento describe el proceso completo de instalación y puesta en marcha del sistema IOT-Server desde cero. El sistema está compuesto por tres servicios Docker (backend FastAPI, frontend React y caché Valkey) y utiliza un protocolo de Reto Criptográfico (RC) para verificar la autenticidad del backend antes de exponer la interfaz de usuario. Se documentan dos fases de arranque y las opciones disponibles para la configuración inicial.

Índice

1. Descripción general	2
2. Requisitos previos	2
3. FASE 1 — Primer arranque (sin verificación RC)	2
3.1. Paso 1 — Levantar los contenedores	2
3.2. Paso 2 — Crear la Application (elige una opción)	3
3.2.1. Opción A — Configuración inicial desde el frontend	3
3.2.2. Opción B — Configuración desde Swagger	4
3.3. Paso 3 — Calcular el server_key	5
4. FASE 2 — Segundo arranque (con verificación RC activa)	6
4.1. Paso 4 — Editar docker-compose.yml	6
4.2. Paso 5 — Reconstruir el frontend e iniciar	6
4.3. Verificación final	6
5. Referencia rápida	6
5.1. Resumen del flujo completo	6
5.2. Tabla de variables de configuración	7
5.3. Comportamiento del BackendGate	7
5.4. Reinicio limpio	8
A. Arranque Inicial — Inicio Practico	9

1. Descripción general

El sistema IOT-Server está compuesto por tres servicios Docker que trabajan en conjunto:

Contenedor	Puerto	Tecnología	Descripción
iot-valkey	6379	Valkey 8	Caché de sesiones (Redis-compatible)
iot-backend	8000	FastAPI + SQLite	API REST del sistema IoT
iot-frontend	3000	React + Nginx	Interfaz web con reverse proxy

Cuadro 1: Servicios Docker del sistema IOT-Server

El frontend incluye un **Reto Criptográfico (RC / puzzle)** implementado con AES-256-CBC y HMAC-SHA256 que verifica la autenticidad del backend antes de mostrar la aplicación. Este mecanismo requiere tres valores (`APPLICATION_ID`, `API_KEY` y `SERVER_KEY`) que solo existen una vez que se ha registrado una *Application* en el backend.

Por esta razón, el proceso de arranque se divide en **dos fases**.

2. Requisitos previos

Antes de comenzar, asegúrate de contar con lo siguiente:

- **Docker Desktop** instalado y en ejecución.
- **Python 3** disponible en terminal (para calcular el `server_key` en el Paso 3, si se usa la Opción B).
- Repositorio clonado localmente.
- Archivo `IOT-Server/.env.docker` presente (ya incluido en el repositorio).

3. FASE 1 — Primer arranque (sin verificación RC)

En esta fase el frontend arranca sin los valores RC configurados. El componente `BackendGate` detecta las variables vacías y **omite la verificación**, permitiendo el acceso. Esto es intencional para poder crear la *Application* y obtener las credenciales.

3.1. Paso 1 — Levantar los contenedores

Desde la carpeta `IOT-Server/`, ejecutar:

Listing 1: Comando de arranque inicial

```
cd IOT-Server
docker compose up -d
```

Al arrancar, el backend ejecuta automáticamente:

1. Creación de todas las tablas de la base de datos (`iot.db`).
2. Seed del administrador master inicial.

Las credenciales del administrador master creadas automáticamente son:

Campo	Valor
Email	admin@iot.com
Password	Admin1234!

Para verificar que los tres servicios estén en funcionamiento:

Listing 2: Verificar estado de contenedores

```
docker compose ps
```

Los tres contenedores deben aparecer como **healthy** o **running**. Si alguno aparece como **starting**, espera unos segundos y repite el comando.

3.2. Paso 2 — Crear la Application (elige una opción)

Necesitas crear una *Application* en el backend para obtener las credenciales RC. Hay dos formas de hacerlo:

	Opción A — Frontend (recomendada)	Opción B — Swagger
Interfaz	Visual, con botones de copia al portapapeles	API REST manual
Acceso	localhost:3000	localhost:8000/docs
Ventaja	No requiere cálculos manuales; el panel copia los valores directamente	Control total de los parámetros enviados

Cuadro 2: Comparación de métodos de configuración inicial

3.2.1. Opción A — Configuración inicial desde el frontend

1. Abrir el navegador en `http://localhost:3000`.
2. Iniciar sesión con las credenciales del admin master: **admin@iot.com** / **Admin1234!**
3. El menú lateral solo mostrará la sección **Aplicaciones** (modo setup activo).
4. Aparecerá una alerta azul pidiendo que crees tu primera *Application*. Hacer clic en **Nuevo**.
5. Rellenar el formulario:
 - **Nombre:** Frontend IoT (o el que prefieras)
 - **Descripción:** (opcional)
 - El campo `administrator_id` se rellena automáticamente
6. Confirmar. Aparecerá un **panel amarillo** con las credenciales listas para copiar:
 - Botón de copia para `VITE_APP_APPLICATION_ID`

- Botón de copia para `VITE_APP_API_KEY`
- Comando para obtener `VITE_APP_SERVER_KEY` desde terminal

IMPORTANTE: El `api_key` solo se muestra en este panel mientras estás en modo setup. Si cierras sesión o reconstruyes el frontend sin copiarlo, no podrás recuperarlo y deberás crear una nueva Application.

Continúa en el **Paso 3 — Calcular el server_key**.

3.2.2. Opción B — Configuración desde Swagger

Paso B.1 — Abrir Swagger Navegar a `http://localhost:8000/docs`.

Paso B.2 — Autenticarse

1. Buscar el endpoint `POST /api/v1/auth/login`.
2. Hacer clic en **Try it out** y ejecutar con:

Listing 3: Credenciales de login

```
{
  "email": "admin@iot.com",
  "password": "Admin1234!"
}
```

3. Copiar el valor del campo `access_token` de la respuesta.
4. Hacer clic en el botón **Authorize** () en la parte superior derecha.
5. Pegar `Bearer <access_token>` en el campo y confirmar.

Paso B.3 — Obtener UUID del administrador

1. Buscar el endpoint `GET /api/v1/administrators/`.
2. Ejecutar sin parámetros.
3. Copiar el campo `id` del administrador master.

Paso B.4 — Crear la Application

1. Buscar el endpoint `POST /api/v1/applications`.
2. Ejecutar con:

Listing 4: Body para crear Application

```
{
  "name": "Frontend IoT",
  "description": "Aplicacion web del frontend",
  "administrator_id": "<UUID copiado en B.3>"
}
```

La respuesta incluirá:

Listing 5: Respuesta del endpoint POST /applications

```
{
  "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "name": "Frontend IoT",
  "api_key": "aaaa1111bbbb2222....(64 caracteres hexadecimales)...",
  "is_active": true
}
```

IMPORTANTE: Guarda id y api_key en este momento. El api_key no se puede recuperar después — si se pierde, hay que crear una nueva Application.

Variable	Valor a guardar
VITE_APP_APPLICATION_ID	Campo id del response
VITE_APP_API_KEY	Campo api_key del response

Cuadro 3: Valores a anotar de la respuesta

3.3. Paso 3 — Calcular el server_key

El server_key no lo devuelve el backend directamente. Se deriva de la SECRET_KEY del backend con la siguiente fórmula:

$$\text{server_key} = \text{SHA} - 256(\text{SECRET_KEY} \parallel \text{"|puzzle_v1"})$$

La SECRET_KEY actual se encuentra en IOT-Server/.env.docker:

Listing 6: Valor de SECRET_KEY en .env.docker

```
SECRET_KEY=542
ec75cc3c11285d3134f3aee7d6bcf27292fc97090ec93c99030312ca21e10
```

Calcular con Python desde la terminal:

Listing 7: Cálculo del server_key

```
python -c "import hashlib; sk='542
ec75cc3c11285d3134f3aee7d6bcf27292fc97090ec93c99030312ca21e10';
print(hashlib.sha256((sk+'|puzzle_v1').encode()).hexdigest())"
```

El resultado (64 caracteres hexadecimales) es el valor de VITE_APP_SERVER_KEY. El panel amarillo del frontend (Opción A) muestra directamente el comando para ejecutar dentro del contenedor.

Si cambias SECRET_KEY en .env.docker, debes recalculas este valor antes de reconstruir el frontend.

4. FASE 2 — Segundo arranque (con verificación RC activa)

Con los tres valores obtenidos en la Fase 1, se configura el frontend para ejecutar el puzzle RC en cada carga de la aplicación.

4.1. Paso 4 — Editar docker-compose.yml

Abrir el archivo IOT-Server/docker-compose.yml y reemplazar los valores vacíos en la sección frontend >build >args:

Listing 8: Sección frontend en docker-compose.yml

```
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile.test
    args:
      VITE_API_BASE_URL: /api/v1/
      VITE_APP_APPLICATION_ID: "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx"
  "
    VITE_APP_API_KEY: "aaaa1111bbbb2222....(64 chars hex)...."
    VITE_APP_SERVER_KEY: "resultado del Paso 3"
```

4.2. Paso 5 — Reconstruir el frontend e iniciar

Listing 9: Reconstrucción del frontend

```
docker compose build frontend
docker compose up -d
```

Solo se reconstruye el frontend. El backend y Valkey no necesitan reconstruirse ya que sus datos persisten en los volúmenes Docker.

4.3. Verificación final

Abrir en el navegador: <http://localhost:3000>

El BackendGate ejecutará el puzzle RC antes de mostrar la aplicación:

- Si el backend responde correctamente → la app carga con normalidad.
- Si las claves son incorrectas o el backend no puede validar → la app muestra un error de verificación y **bloquea el acceso**.

5. Referencia rápida

5.1. Resumen del flujo completo

Listing 10: Diagrama del flujo de instalación

```

FASE 1 -- Sin RC
-----
Paso 1: docker compose up -d
|
Paso 2 (elige una):
  Opcion A -> localhost:3000 -> login -> Aplicaciones -> Nuevo
              panel amarillo con botones de copia
  Opcion B -> localhost:8000/docs -> login -> GET /administrators
              -> POST /applications -> copiar id + api_key
|
Paso 3: python SHA-256(SECRET_KEY + "|puzzle_v1") -> server_key

FASE 2 -- Con RC activo
-----
Paso 4: Editar docker-compose.yml -> pegar los 3 valores VITE_APP_*
|
Paso 5: docker compose build frontend
        docker compose up -d
|
http://localhost:3000 -> BackendGate verifica puzzle RC OK

```

5.2. Tabla de variables de configuración

Variable	Origen	Dónde se configura
VITE_API_BASE_URL	Fijo: /api/v1/	docker-compose.yml
VITE_APP_APPLICATION_ID	Campo id de POST /applications	docker-compose.yml
VITE_APP_API_KEY	Campo api_key de POST /applications	docker-compose.yml
VITE_APP_SERVER_KEY	SHA-256(SECRET_KEY + " puzzle_v1")	docker-compose.yml
SECRET_KEY	Token hex de 32 bytes (libre)	.env.docker

Cuadro 4: Variables de configuración del sistema

5.3. Comportamiento del BackendGate

Estado de VITE_APP_*	Comportamiento
Las tres variables configuradas	Ejecuta puzzle RC — bloquea si el backend no valida
Alguna variable vacía o ausente	Omite verificación RC — permite acceso (modo setup)

Cuadro 5: Comportamiento del componente BackendGate según configuración

5.4. Reinicio limpio

Si necesitas destruir todos los datos y volver al estado inicial:

Listing 11: Comando de reinicio limpio

```
docker compose down -v --remove-orphans
```

Esto elimina contenedores, redes y volúmenes (base de datos SQLite y caché Valkey). Después de esto repite desde el **Paso 1**.

Importante: Recuerda vaciar las variables RC en `docker-compose.yml` (dejarlas como cadenas vacías) antes de volver a levantar. Si se dejan los valores de la instalación anterior, el frontend intentará el puzzle con credenciales inexistentes y bloqueará el acceso permanentemente.

A. Arranque Inicial — Inicio Practico

Este anexo contiene las capturas de pantalla del proceso de arranque inicial del sistema, documentando cada paso con evidencia visual.

Ilustración 1 — Ejecución de `docker compose up -d`

Desde la carpeta raíz del proyecto se ejecuta el comando de arranque. Docker construye las tres imágenes y las levanta secuencialmente.

```
PS C:\Users\gemma\OneDrive\Documents\MCTS\Desarrollo de software seguro\Proyecto Fronted\IOT-Server> docker compose up -d
[+] up 7/7
  ✓ Image valkey/valkey:8-alpine Pulled 7.1s
#1 [internal] load local bake definitions
#1 reading from stdin 1.46kB 0.0s done
#1 DONE 0.0s

#2 [backend internal] load build definition from Dockerfile
#2 transferring dockerfile: 1.16kB 0.0s done
#2 DONE 0.0s

#3 [frontend internal] load build definition from Dockerfile.test
#3 transferring dockerfile: 772B done
#3 DONE 0.0s

#4 [frontend internal] load metadata for docker.io/library/nginx:1.25-alpine
#4 ...

#5 [backend internal] load metadata for docker.io/library/python:3.12-slim
#5 DONE 1.0s

#6 [frontend internal] load metadata for docker.io/library/node:20-alpine
#6 DONE 1.0s

#4 [frontend internal] load metadata for docker.io/library/nginx:1.25-alpine
#4 DONE 1.0s

#7 [frontend internal] load .dockerignore
#7 transferring context: 96B done
#7 DONE 0.0s

#8 [frontend stage-1 1/3] FROM docker.io/library/nginx:1.25-alpine@sha256:516475cc129da42866742567714ddc681e5eed7b9ee0b9e9c015e464b4221a00
#8 resolve docker.io/library/nginx:1.25-alpine@sha256:516475cc129da42866742567714ddc681e5eed7b9ee0b9e9c015e464b4221a00 0.0s done
#8 DONE 0.0s

#9 [frontend internal] load build context
#9 transferring context: 5.20kB 0.0s done
#9 DONE 0.0s
```

Figura 1: Terminal: inicio de la construcción y arranque de contenedores

```
#33 [frontend] exporting to image
#33 exporting layers
#33 exporting layers 0.2s done
#33 exporting manifest sha256:3977e420a052b5cee6daab4ce9979967a852aa9daaae68850030d38cee5fe78f 0.0s done
#33 exporting config sha256:c1db8bb0bb8159bd9a3016be95951bafbac7e253e24bf12e4c601ab49223c9fa 0.0s done
#33 exporting attestation manifest sha256:18bf4468f88bf9f56cc04136051ad888455ff055109d7c8c4a3cf9e79dce29c0 0.0s done
#33 exporting manifest list sha256:96660041774504cb53feea9d6af886dfcbbfc27bbc53c07bb424fdef77e3c1a20 0.0s done
#33 exporting manifest list sha256:96660041774504cb53feea9d6af886dfcbbfc27bbc53c07bb424fdef77e3c1a20 0.0s done
#33 naming to docker.io/library/iot-server-frontend:latest done
#33 unpacking to docker.io/library/iot-server-frontend:latest 0.1s done
#33 DONE 0.5s

[+] up 15/15d resolving provenance for metadata file
  ✓ Image valkey/valkey:8-alpine Pulled 7.1s
  ✓ Image iot-server-frontend Built 70.3s
  ✓ Image iot-server-backend Built 70.3s
  ✓ Network iot-server_default Created 0.1s
  ✓ Volume iot-server_sqlite_data Created 0.1s
  ✓ Volume iot-server_valkey_data Created 0.0s
  ✓ Container iot-valkey Healthy 6.2s
  ✓ Container iot-backend Healthy 17.5s
  ✓ Container iot-frontend Started 17.9s
```

Figura 2: Terminal: construcción completada — los tres contenedores están en ejecución

Ilustración 2 — Los tres contenedores en Docker Desktop

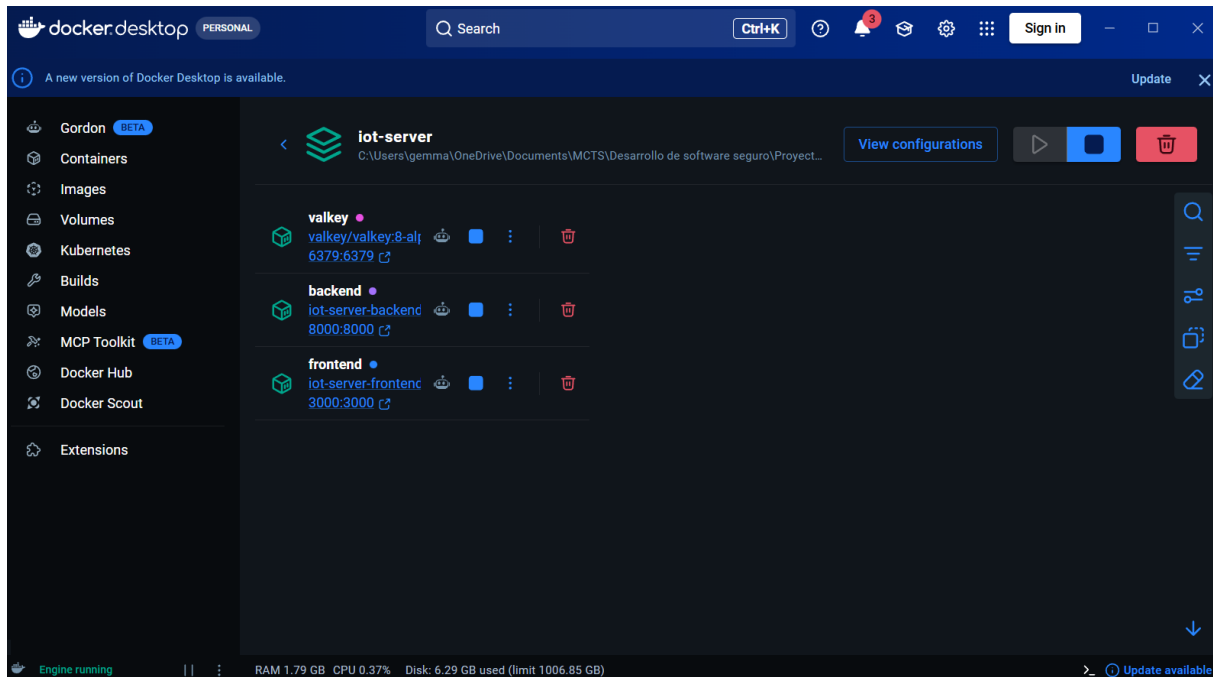


Figura 3: Docker Desktop: `iot-valkey`, `iot-backend` e `iot-frontend` activos

Ilustración 3 — Acceso al login del admin master

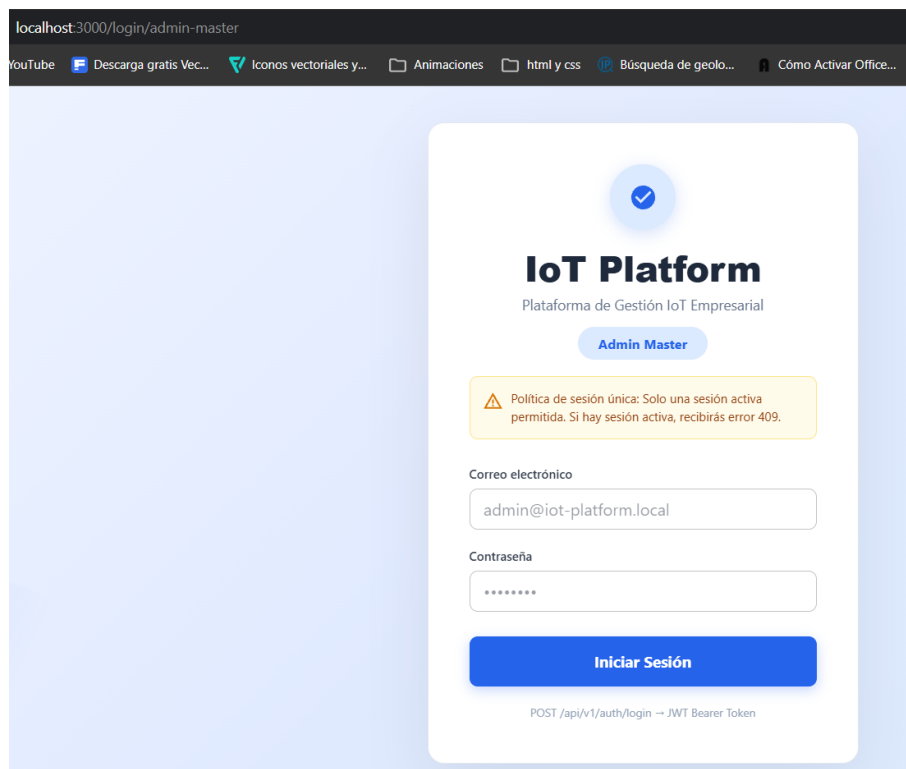


Figura 4: Pantalla de login del admin master en `localhost:3000`

Ilustraciones 4 y 5 — Modo de configuración inicial

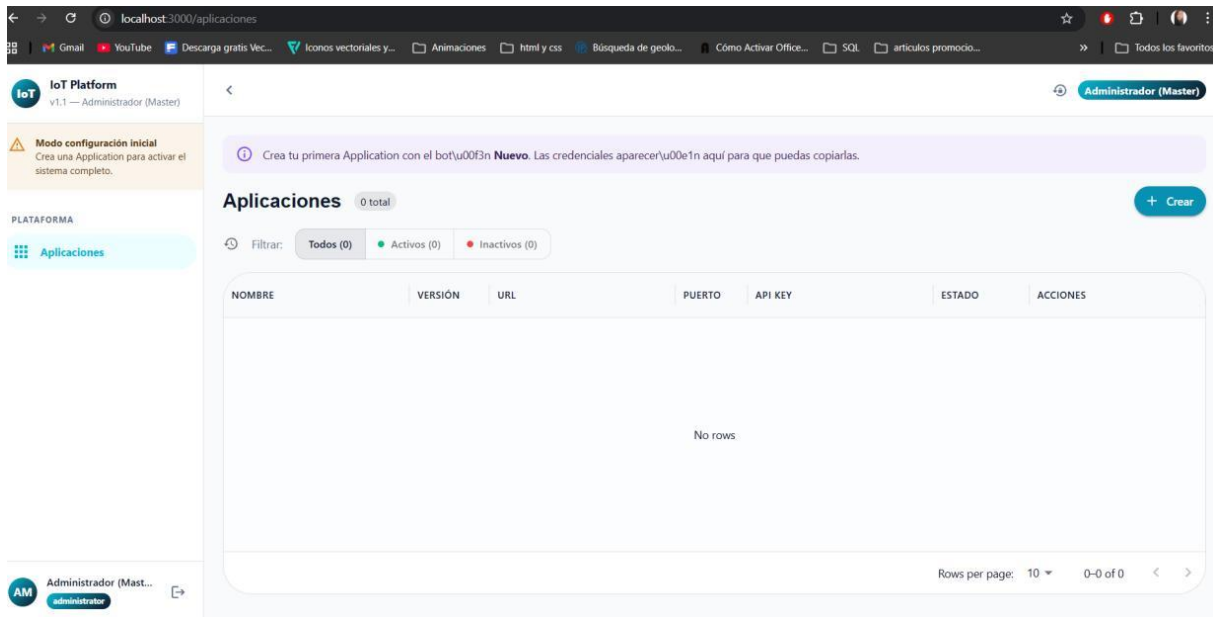


Figura 5: Configuraci\u00f3n inicial: el men\u00fa solo permite gestionar Aplicaciones

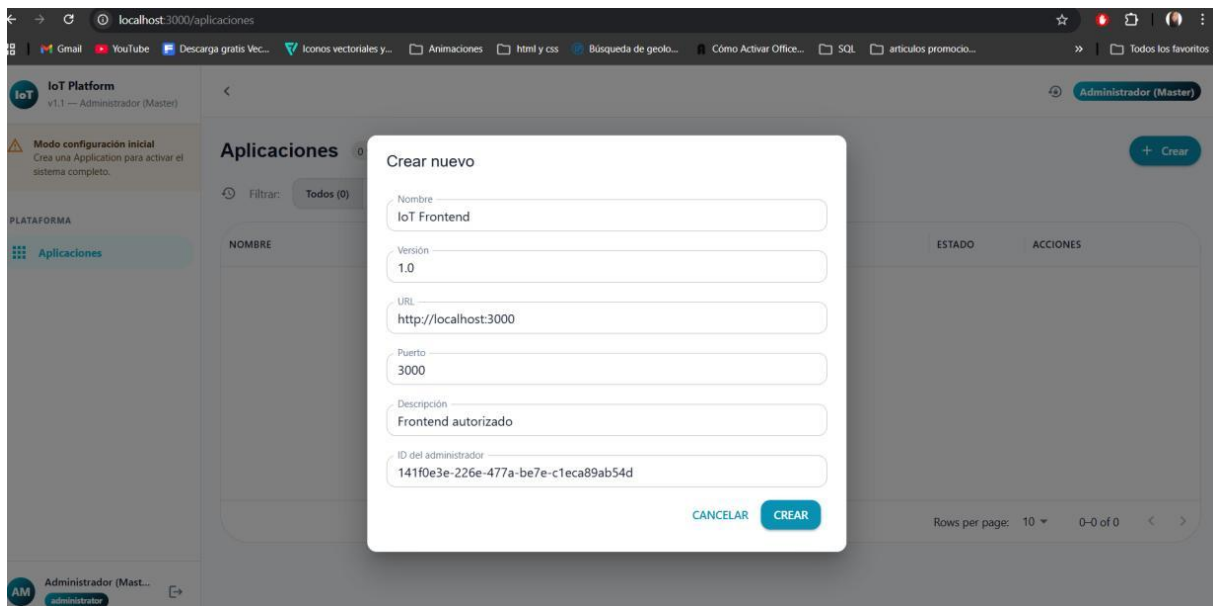


Figura 6: Formulario de creaci\u00f3n de la primera Application

Ilustraciones 6, 7 y 8 — Obtención de credenciales RC

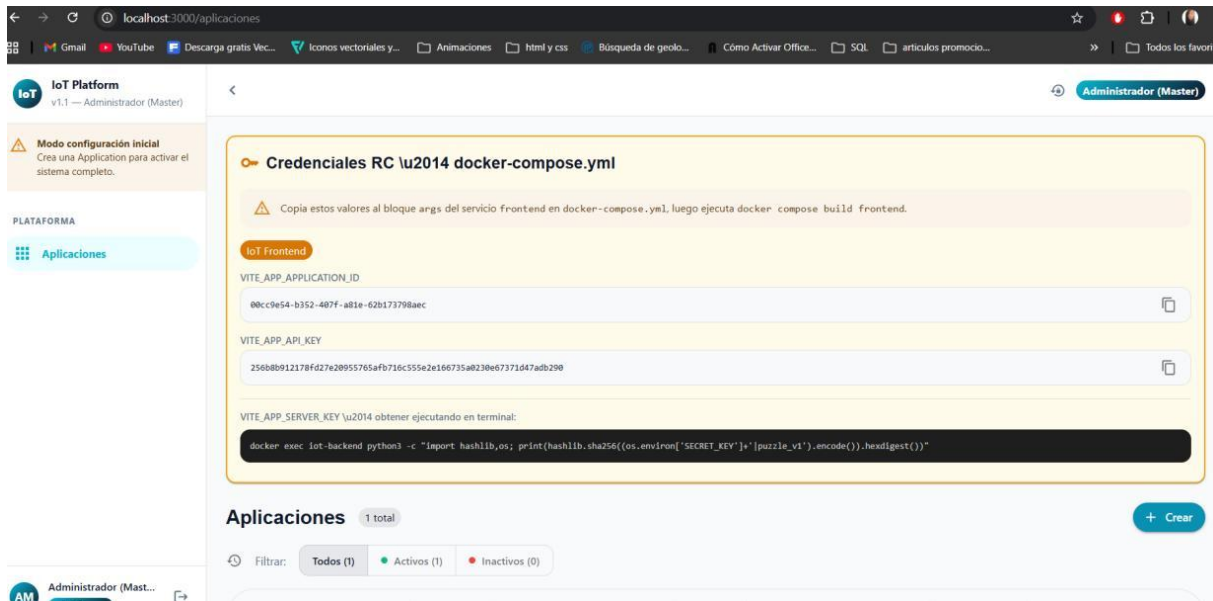


Figura 7: Panel amarillo con credenciales RC: APPLICATION_ID y API_KEY listos para copiar

```
PS C:\Users\gemma\OneDrive\Documents\MCTS\Desarrollo de software seguro\Proyecto Frontend\IOT-Server> docker exec iot-backend python3 -c "import hashlib,os; print(hashlib.sha256((os.environ['SECRET_KEY']+'|puzzle_v1').encode()).hexdigest())"
8269a34f97e745a6c2b7df5be5184f76f878f647d356cfe46a10bf0022c680d4
```

Figura 8: Derivación del server_key ejecutando el comando SHA-256

```
Run Service
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile.test
  args:
    # URL relativa: nginx hace proxy al backend
    VITE_API_BASE_URL: /api/v1/
    # Credenciales RC – generadas al crear la Application en setup mode
    # Dejar vacías para entrar en modo configuración inicial
    VITE_APP_APPLICATION_ID: "35adda83-8ba5-4787-ada4-47e1fd977196"
    VITE_APP_API_KEY: "00e8b86d0abb94c51280f8bb63a439e7d4ee3c51ce777531fec15b274d428e22"
    VITE_APP_SERVER_KEY: "8269a34f97e745a6c2b7df5be5184f76f878f647d356cfe46a10bf0022c680d4"
  container_name: iot-frontend
  ports:
    - "3000:3000"
```

Figura 9: Inserción de las credenciales en docker-compose.yml

Ilustraciones 9, 10 y 11 — Reconstrucción y verificación

```
PS C:\Users\gemma\OneDrive\Documents\MCTS\Desarrollo de software seguro\Proyecto Fronted\IOT-Server> docker compose build frontend
#1 [internal] load local bake definitions
#1 reading from stdin 1.60kB 0.0s done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile.test
#2 transferring dockerfile: 772B 0.0s done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/node:20-alpine
#3 DONE 1.2s

#4 [internal] load metadata for docker.io/library/nginx:1.25-alpine
#4 DONE 1.3s

#5 [internal] load .dockerignore
#5 transferring context: 96B done
#5 DONE 0.0s

#6 [stage-1 1/3] FROM docker.io/library/nginx:1.25-alpine@sha256:516475cc129da42866742567714ddc681e5eed7b9ee0b9e9c015e464b4221a00
#6 resolve docker.io/library/nginx:1.25-alpine@sha256:516475cc129da42866742567714ddc681e5eed7b9ee0b9e9c015e464b4221a00 0.0s done
#6 DONE 0.0s

#7 [build 1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293
#7 resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 0.0s done
#7 DONE 0.1s
```

Figura 10: Ejecución de `docker compose build frontend` para salir del modo setup

```
PS C:\Users\gemma\OneDrive\Documents\MCTS\Desarrollo de software seguro\Proyecto Fronted\IOT-Server> docker compose up -d
[+] up 3/3
✓Container iot-valkey Healthy
✓Container iot-backend Healthy
✓Container iot-frontend Started
```

Figura 11: Estado de los contenedores tras la reconstrucción

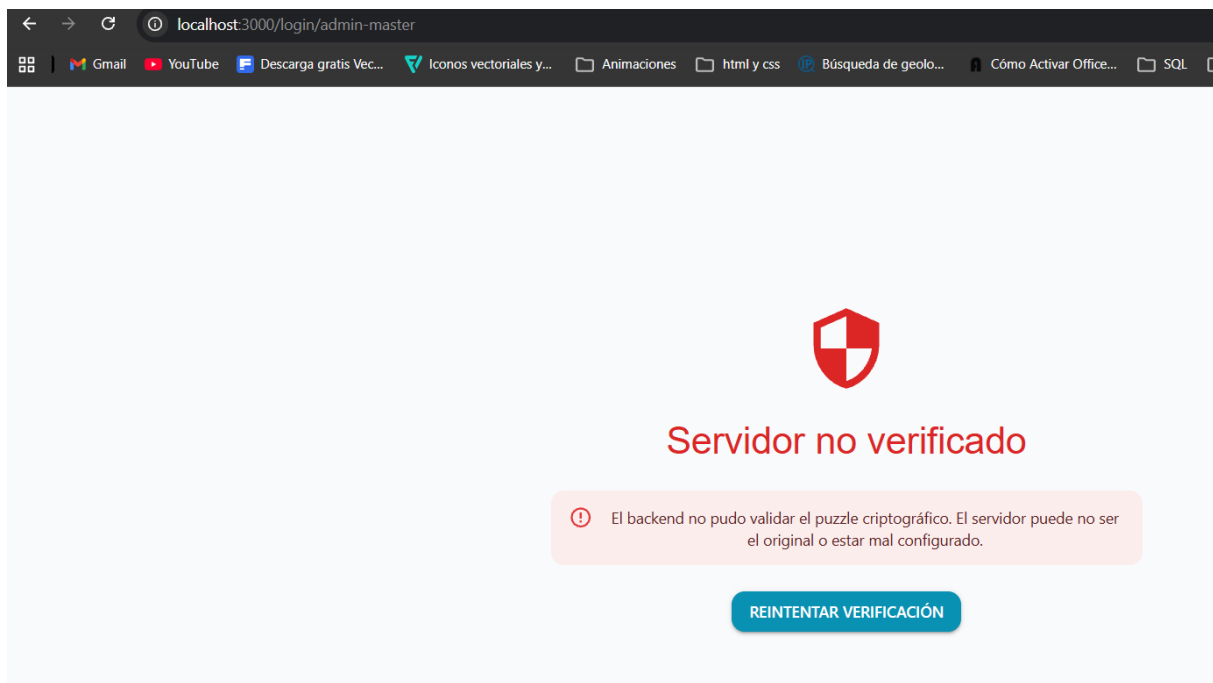


Figura 12: Prueba de seguridad: credenciales falsas bloquean el lanzamiento del frontend

Ilustración 12 — Frontend lanzado correctamente

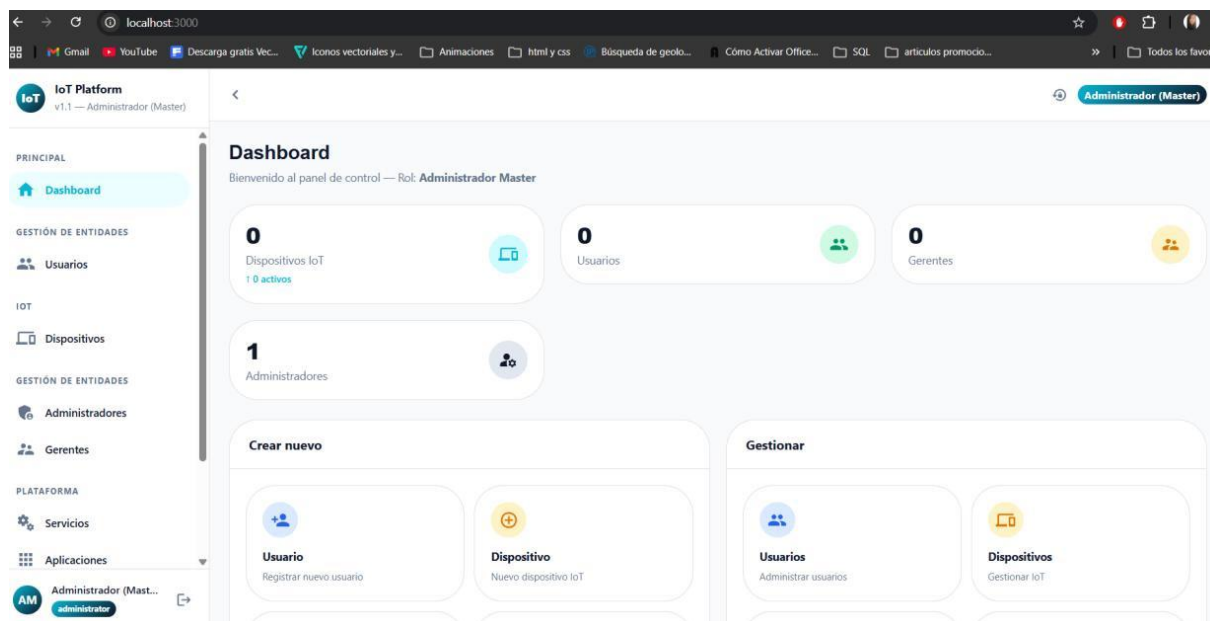


Figura 13: El frontend se lanza correctamente con la verificación RC superada